



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE THREE: JAVA FUNDAMENTALS

UNIT TWO: JAVA BASICS - DATA TYPES, VARIABLES AND OPERATORS IN JAVA PART II

JAVA BASICS - DATA TYPES, VARIABLES AND OPERATORS IN JAVA PART II

Learning Objectives

By the end of this unit, you will be able to:

- **declare, initialize, and use variables following Java naming conventions.**
- **apply arithmetic, relational, logical, and assignment operators in Java programs.**
- **declare constants using the final keyword.**
- **explain the concept of operator precedence in Java.**
- **write and execute Java programs that demonstrate correct use of data types, variables, and operators.**

Learning Outcomes

At the end of this unit, you should be able to:

- **declare, initialize and use variables while adhering to Java naming convention.**
- **apply arithmetic, relational, logical and assignment operators in Java programs.**
- **understand how to declare constant using the final keyword.**
- **understand operator precedence.**
- **write and execute Java programs that demonstrate the proper use of data types, variables and operators.**

Introduction

- **This unit delves into detailed aspects of data types (including primitive types like int, double, boolean and reference types like String and arrays), variables (covering declaration and initialization), constants (using the final keyword), and operators (arithmetic, relational, logical, assignment, and their precedence rules).**
- **You will learn how to declare and initialize variables, understand variable scope, and understand the roles of operators in performing calculations, comparisons, and logical decisions.**

Usage of Data Types in Java

Each data type serves a specific purpose. Consider the code sample below:

```
public class PrimitiveTypes {  
    public static void main (String[] args) {  
        int age = 30;  
        double salary = 85999.90;  
        char grade = 'B';  
        boolean married = false;  
  
        System.out.println("Age: " + age);  
        System.out.println("Salary: $" + salary);  
        System.out.println("Grade: " + grade);  
        System.out.println("Married: " + married);  
    }  
}
```

Usage of Variables and Constants

A variable is a named memory location used to store a value. It must be declared with a data type before use. The syntax is:

dataType **variableName** = value;

Examples:

int **age** = 25;

// Integer variable

String **name** = "John";

// String variable

boolean **isActive** = true;

// Boolean variable

double **price** = 19.99;

// Double variable

Constants

A constant is a data container whose value cannot change once it has been assigned. Constants improve code readability and prevent accidental modifications. In Java, constants are declared using the *final* keyword.

Syntax is:

final dataType CONSTANT_NAME = value;

Examples

final double PI = 3.14159;

final int MAX_USERS = 100;

final String COMPANY_NAME = "TechCorp"

Example of Variables and Constants (1)

The TaxInclusivePricing class calculates the total price of items including tax. It uses a constant tax rate of 15%. In the main method, the total is computed by multiplying the quantity, the base price and the tax rate.

```
public class TaxInclusivePricing {  
    // Constant (static final)  
    public static final double TAX_RATE = 0.15;  
    public static void main(String[] args) {  
        // Variables  
        int quantity = 10;  
        double price = 29.99;  
        double total = price * quantity * (1 + TAX_RATE);  
        System.out.println("Total cost: $" + total);  
    }  
}
```

Example of Variables and Constants (2)

```
public class BankAccount {  
    // Constant  
    public static final double INTEREST_RATE = 0.05;  
    public static void main(String[] args) {  
        // Variables  
        double balance = 1000.0;  
        double interestEarned = balance * INTEREST_RATE;  
  
        System.out.println("Interest Earned: $" + interestEarned);  
  
        // Deposit more money  
        balance = balance + 500;  
        interestEarned = balance * INTEREST_RATE;  
        System.out.println("New Interest: $" + interestEarned);  
    }  
}
```

Example of Variables and Constants (2)

- In the BankAccount class, a constant, `INTEREST_RATE` of 0.05 (5%) was declared and initialized.
- In the main method, balance was initialized with \$1000.0 and the interest earned was calculated using the formula:

$$\text{interestEarned} = \text{balance} * \text{INTEREST_RATE}.$$

- The result is printed.
- After depositing an additional \$500 into the balance, the program recalculates the interest on the new balance and prints the updated interest.

Example of Variables and Constants (3)

```
public class TemperatureConverter {  
    // Constants for conversion  
    static final double FAH_FACTOR = 9.0 / 5;  
    static final double FAH_OFFSET = 32;  
    public static void main(String[] args) {  
        // Initial value for celsius  
        double celsius = 25.0;  
        double fahrenheit = (celsius * FAH_FACTOR) + FAH_OFFSET;  
        System.out.println(celsius + "C = " + fahrenheit + "F");  
  
        // Change the Value for Celsius  
        celsius = 30;  
        fahrenheit = (celsius * FAH_FACTOR) + FAH_OFFSET;  
        System.out.println(celsius + "C = " + fahrenheit + "F");  
    }  
}
```

Example of Variables and Constants (3)

- In the TemperatureConverter class, temperature is converted from Celsius to Fahrenheit
- Two constants, ***FAH_FACTOR (9.0/5)*** and ***FAH_OFFSET (32)***, which represent the multiplier and offset in the conversion formula are declared and initialized.
- In the main method, the program initializes a variable celsius with a value of 25.0, calculates the corresponding Fahrenheit temperature, and prints the result.
- The value of celsius is updated to 30, the new value of Fahrenheit was recalculated, and the new conversion is printed.

Operators - Arithmetic Operators

Operator	Description	Example (a = 7, b = 2, x = 18.0, y = 9.0)	Result	Notes
+	Addition	$a + b = 7 + 2$	9	Works with int, long, float and double
-	Subtraction	$a - b = 7 - 2$	5	Works with all numeric types
*	Multiplication	$a * b = 7 * 2$	14	Works with all numeric types
/	Integer Division	$a/b = 7/2$	3	Truncates decimal part
	Floating-Point Division	$x/y = 18.0/9.0$	2.0	Keeps decimal precision
%	Modulus	$a \% b = 7\%2$	1	Returns remainder
		$x \% y = 18.0 \% 9.0$	0.0	Works with floating point
		$-8 \% 3$	-2	Sign follows dividend

Operators - Arithmetic Operators

The following are common pitfalls with arithmetic operators

- 1. Division by Zero**
- 2. Modulus with negative numbers**
- 3. String concatenation**

Operators - Arithmetic Operators

```
1  public class ArithmeticOperatorsPitfalls {  
2      public static void main(String[] args) {  
3          System.out.println(10/0);  
4          System.out.println(10.0/0);  
5          System.out.println(-7%3);  
6          System.out.println(7%-3);  
7      }  
8  }
```

In the code above :

Line 3 will throw an ArithmeticException error.

Line 4: Infinity

Line 5: -1

Line 6: 1

Relational (Comparative) Operators

Operator	Description	Example (a = 5, b = 3)	Return Value	Notes
==	Equal to	a == b	false	Compares values (for primitives) or references (for objects)
!=	Not equal to	a != b	true	Opposite of ==
>	Greater than	a > b	true	Works with numbers
<	Lesser than	a < b	false	Works with numbers
>=	Greater than or equal to	a >= b	true	true, if either > or ==
<=	Lesser than or equal to	a <= b	false	true, if either < or ==

Usage of Relational Operators

Example 1: Primitive comparison

```
public class PrimitiveComparison {  
    public static void main(String[] args) {  
        int y = 5;  
        int x = 5;  
        double a = 0.1 + 0.2;  
        double b = 0.3;  
        System.out.println(y == x); // true  
        System.out.println(a == b); // false  
    }  
}
```

In the code above, the comparison between a and b is false because $0.1 + 0.2 = 0.30000000000000004$.

Usage of Relational Operators

Example 2: Comparing different data types

```
1  public class CompareDifferentDataTypes {  
2      public static void main(String[] args) {  
3          int num = 10;  
4          double decimal = 10.0;  
5          char letter = 'A';  
6          System.out.println(num == decimal);  
7          System.out.println(letter > num);  
8      }  
9  }
```

Line 6 outputs true because `==` compares values, not types, 10.0 (double) equals 10 (int). Line 7 is also true since 'A' has an ASCII value of 65, which is greater than 10.

Logical Operators

These operators determine the truth value of a condition or combined conditions, returning either true or false (See the Table below).

Operator	Name	Description	Example (a = true, b = false)
&&	Logical AND	Returns true only if both operands are true	a && b = false
	Logical OR	Returns true if at least one operand is true	a b = true
!	Logical NOT	Reverses the boolean value	!a = false
XOR	Logical XOR (Exclusive OR)	Returns true if only one operand is true	a ^ b = true

Usage of Logical Operators

Example 1: Boolean with relational operators

```
1  public class BooleanWithRelational {  
2      public static void main(String[] args) {  
3          int x = 7;  
4          int y = 12;  
5  
6          System.out.println(x < 10 && y > 10);  
7          System.out.println(x == y || y != 0);  
8          System.out.println(x == y && y != 0);  
9      }  
10 }
```

Line 6 is true as x is less than 10 and y is greater than 12. Line 7 is true because y is not equal 0. Line 8 is false because while y is not equal to 0 is true, x is equal to y is false.

Usage of Logical Operators

Example 2: Boolean with relational operators

```
1  public class AdulthoodDriver {
2      public static void main(String[] args) {
3          boolean isAdult = true;
4          boolean hasLicense = false;
5
6          if (isAdult && hasLicense) {
7              System.out.println("Can drive");
8          } else {
9              System.out.println("Cannot drive");
10         }
11     }
12 }
```

In the code above, line 7 is not executed. This is because the condition in line 6 is false. However, line 9 executes.

In-Text Questions (ITQs)

- 1. What is the syntax to declare and initialize a variable in Java?**
- 2. How do logical operators affect conditional statements?**
- 3. How can parentheses change the evaluation of expressions?**
- 4. Why is operator precedence important in Java expressions?**

In-Text Answers (ITAs)

1. The syntax is: ***dataType variableName = value;***

For example, `int age = 25;` declares an integer variable named age with the value 25.

2. Logical operators combine multiple conditions in if-statements, controlling program flow by evaluating the overall truth value.
3. Parentheses override default precedence, forcing Java to evaluate the expressions inside them first.
4. Operator precedence determines the order in which parts of an expression are evaluated, affecting the final result.

Conclusion

- **This unit provided the foundational knowledge needed to declare and initialize variables, enforce immutability with constants, and apply a variety of operators correctly.**
- **The ability to handle arithmetic, relational, and logical operations confidently supports making decisions and controlling program behavior, skills that are indispensable as you progress in Java development.**
- **As you continue your learning journey, these core concepts will serve as the groundwork for building more sophisticated applications and solving complex programming problems.**

Summary

- **The unit covered constants using the final keyword. Real-world examples, like tax, temperature, and interest calculations, demonstrated the use of variables and constants.**
- **It also explored arithmetic, relational, and logical operators. You learned how arithmetic operators handle calculations, including integer vs. floating-point division, and common issues like division by zero.**
- **Relational operators were used for comparisons, returning boolean values essential for decision-making. Logical operators showed how to combine conditions to guide program flow.**